

LECTURE 3

RECURSION

OBJECTIVES

- Learn about recursive definitions
 - Explore the base case and the general case of a recursive definition
 - Learn about recursive algorithm
 - Learn about recursive functions
 - Explore how to use recursive functions to implement recursive
-

AGENDA

3.1 Introduction to Recursion

3.2 Recursive Functions

3.3 Classic Recursive Examples

3.4 Recursion vs. Iteration

3.5 Backtracking (8-Queens Puzzle)

3.1. RECURSIVE DEFINITIONS

- Recursion
 - Process of solving a problem by reducing it to smaller versions of itself
- Example: factorial problem
 - 5!
 - $5 \times 4 \times 3 \times 2 \times 1 = 120$
 - If n is a nonnegative
 - Factorial of n ($n!$) defined as follows:

$$0! = 1 \quad \text{(Equation 6-1)}$$

$$n! = n \times (n - 1)! \quad \text{if } n > 0 \quad \text{(Equation 6-2)}$$

3.1. RECURSIVE DEFINITIONS (CONT.)

- Direct solution (Equation 6-1)
 - Right side of the equation contains no factorial notation
- Recursive definition
 - A definition in which something is defined in terms of a smaller version of itself
- Base case (Equation 6-1)
 - Case for which the solution is obtained directly
- General case (Equation 6-2)
 - Case for which the solution is obtained indirectly using recursion

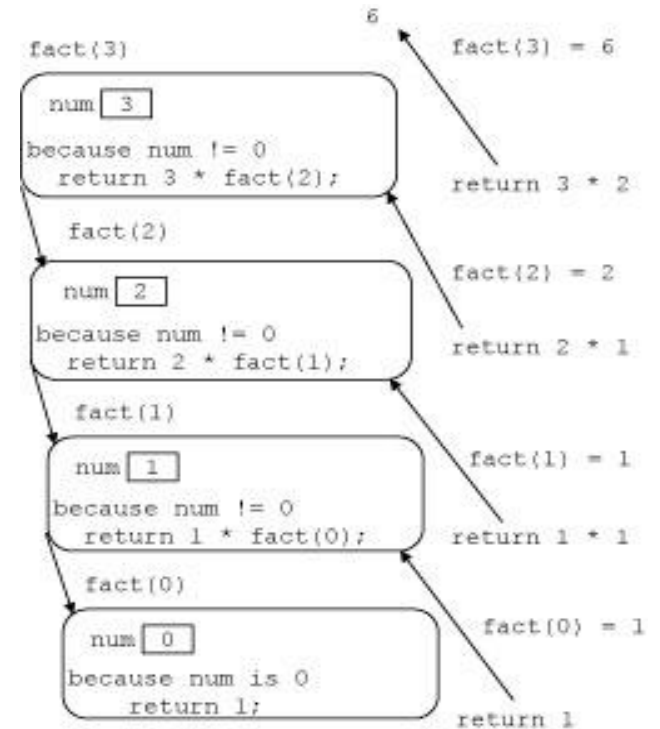
3.1. RECURSIVE DEFINITIONS (CONT.)

- Recursion insight gained from factorial problem
 - Every recursive definition must have one (or more) base cases
 - General case must eventually reduce to a base case
 - Base case stops recursion
- Recursive algorithm
 - Finds problem solution by reducing problem to smaller versions of itself
- Recursive function
 - Function that calls itself

3.2. RECURSIVE FUNCTION

- Recursive function implementing the factorial function

```
int fact(int num)
{
    if (num == 0)
        return 1;
    else
        return num * fact(num - 1);
}
```



Execution of fact(4)

- Recursive function notable comments
 - Recursive function has unlimited number of copies of itself (logically)
 - Every call to a recursive function has its own
 - Code, set of parameters, local variables
 - After completing a particular recursive call
 - Control goes back to calling environment (previous call)
 - Current (recursive) call must execute completely before control goes back to the previous call
 - Execution in previous call begins from point immediately following the recursive call
-

3.2.1. DIRECT AND INDIRECT RECURSION

- Directly recursive function
 - Calls itself
- Indirectly recursive function
 - Calls another function, eventually results in original function call
 - Requires same analysis as direct recursion
 - Base cases must be identified, appropriate solutions to them provided
 - Tracing can be tedious
- Tail recursive function
 - Last statement executed: the recursive call

3.2.2. INFINITE RECURSION

- Occurs if every recursive call results in another recursive call
- Executes forever (in theory)
- Call requirements for recursive functions
 - System memory for local variables and formal parameters
 - Saving information for transfer back to right caller
- Finite system memory leads to
 - Execution until system runs out of memory
 - Abnormal termination of infinite recursive function

3.2.3. REQUIREMENTS TO DESIGN A RECURSIVE FUNCTION

- Understand problem requirements
- Determine limiting conditions
- Identify base cases, providing direct solution to each base case
- Identify general cases, providing solution to each general case in terms of smaller versions of itself

3.3. CLASSIC RECURSIVE EXAMPLES

3.3.1. FIBONACCI NUMBER

- Sequence: 1, 1, 2, 3, 5, 8, 13, 21, 34 . . .
- Given first two numbers (a_1 and a_2)
 - n th number a_n , $n \geq 3$, of sequence given by: $a_n = a_{n-1} + a_{n-2}$
- Recursive function: `rFibNum`
 - Determines desired Fibonacci number
 - Parameters: three numbers representing first two numbers of the Fibonacci sequence and a number n , the desired n th Fibonacci number
 - Returns the n th Fibonacci number in the sequence

3.3.1.FIBONACCI NUMBER (CONT.)

- Third Fibonacci number
 - Sum of first two Fibonacci numbers
- Fourth Fibonacci number in a sequence
 - Sum of second and third Fibonacci numbers
- Calculating fourth Fibonacci number
 - Add second Fibonacci number and third Fibonacci number

3.3.1.FIBONACCI NUMBER (CONT.)

- Recursive algorithm
 - Calculates n th Fibonacci number
 - a denotes first Fibonacci number
 - b denotes second Fibonacci number
 - n denotes n th Fibonacci number

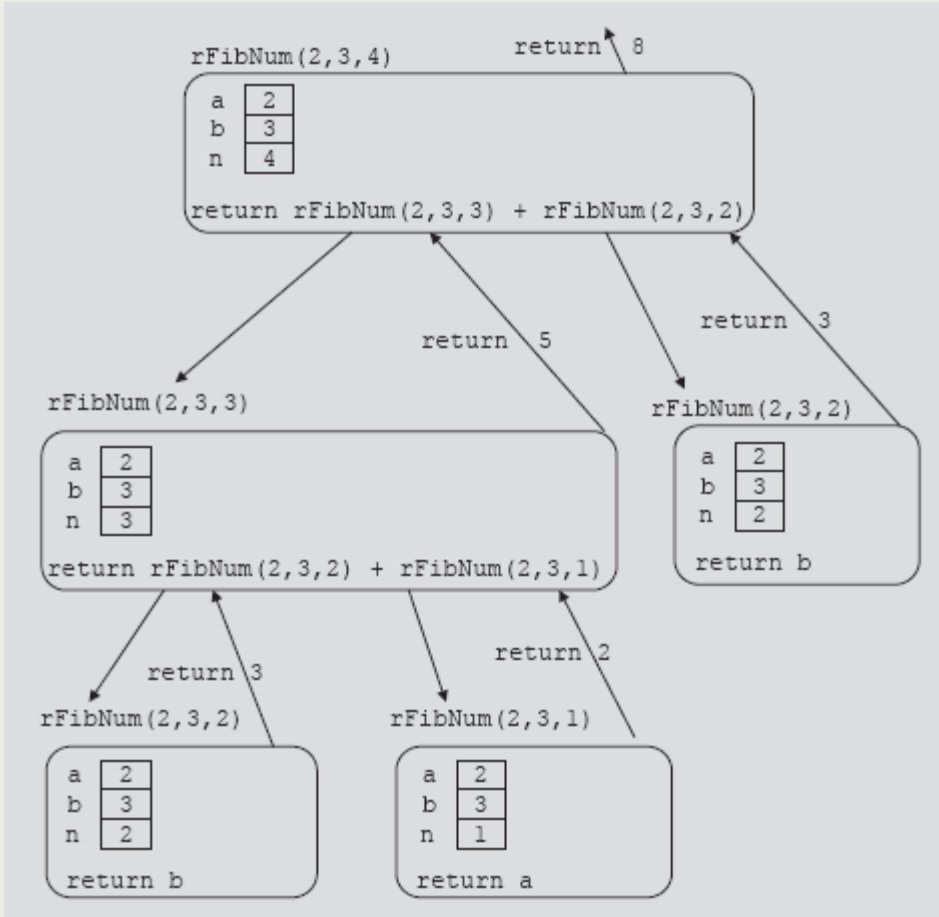
$$rFibNum(a, b, n) = \begin{cases} a & \text{if } n = 1 \\ b & \text{if } n = 2 \\ rFibNum(a, b, n - 1) + rFibNum(a, b, n - 2) & \text{if } n > 2. \end{cases} \quad (\text{Equation 6-3})$$

3.3.1.FIBONACCI NUMBER (CONT.)

- Recursive function implementing algorithm
- Trace code execution
- Review code on page 368 illustrating the function `rFibNum`

```
int rFibNum(int a, int b, int n)
{
    if (n == 1)
        return a;
    else if (n == 2)
        return b;
    else
        return rFibNum(a, b, n - 1) + rFibNum(a, b, n - 2);
}
```

3.3.1.FIBONACCI NUMBER (CONT.)



Execution of `rFibNum(2, 3, 4)`

RECURSION ON DATA STRUCTURES

ARRAYS AND LIST

- Arrays
- Lists

3.3.2.LARGEST ELEMENT IN AN ARRAY

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
list	5	8	2	10	9	4	

list with six elements

- list: array name containing elements
- `list[a]...list[b]` stands for the array elements `list[a]`, `list[a + 1]`, ..., `list[b]`
- `list length = 1`
 - One element (largest)
- `list length > 1`
 - `maximum(list[a],`
`largest(list[a + 1]...list[b]))`

3.3.2.LARGEST ELEMENT IN AN ARRAY (CONT.)

- `maximum(list[0], largest(list[1]...list[5]))`
- `maximum(list[1], largest(list[2]...list[5]), etc.`
- Every time previous formula used to find largest element in a sublist
 - Length of sublist in next call reduced by one

3.3.2.LARGEST ELEMENT IN AN ARRAY (CONT.)

- Recursive algorithm in pseudocode

Base Case: The size of the list is 1

The only element in the list is the largest element

General Case: The size of the list is greater than 1

To find the largest element in `list[a]...list[b]`

1. Find the largest element in `list[a + 1]...list[b]`
and call it `max`
2. Compare the elements `list[a]` and `max`
if (`list[a] >= max`)
the largest element in `list[a]...list[b]` is `list[a]`
otherwise
the largest element in `list[a]...list[b]` is `max`

3.3.2.LARGEST ELEMENT IN AN ARRAY (CONT.)

- Recursive algorithm as a C++ function

```
int largest(const int list[], int lowerIndex, int upperIndex)
{
    int max;

    if (lowerIndex == upperIndex) //size of the sublist is one
        return list[lowerIndex];
    else
    {
        max = largest(list, lowerIndex + 1, upperIndex);

        if (list[lowerIndex] >= max)
            return list[lowerIndex];
        else
            return max;
    }
}
```

3.3.2.LARGEST ELEMENT IN AN ARRAY (CONT.)

- Trace execution of the following statement

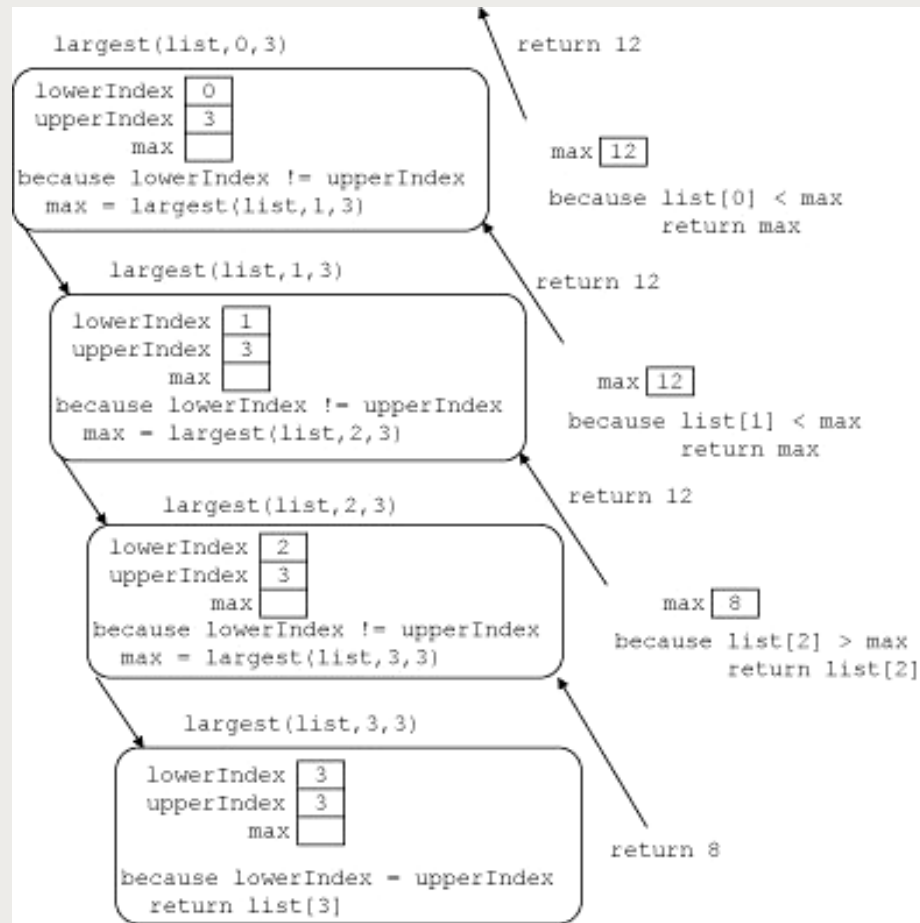
```
cout << largest(list, 0, 3) << endl;
```

- C++ program
 - Determines largest element in a list

	[0]	[1]	[2]	[3]
list	5	10	12	8

list with four elements

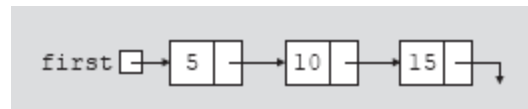
3.3.2.LARGEST ELEMENT IN AN ARRAY (CONT.)



Execution of `largest(list, 0, 3)`

3.3.3 PRINT A LINKED LIST IN REVERSE ORDER

- Function `reversePrint`
 - Given list pointer, prints list elements in reverse order
- Example
 - Links in one direction
 - Cannot traverse backward starting from last node



Linked list

3.3.3.PRINT A LINKED LIST IN REVERSE ORDER (CONT.)

- Cannot print first node info until remainder of list printed
- Cannot print second node info until tail of second node printed, etc.
- Every time tail of a node considered
 - List size reduced by one
 - Eventually list size reduced to zero
 - Recursion stops

3.3.3.PRINT A LINKED LIST IN REVERSE ORDER (CONT.)

- Recursive algorithm in pseudocode

Base Case: List is empty: no action

General Case: List is nonempty

1. Print the tail
2. Print the element

- Recursive algorithm in C++

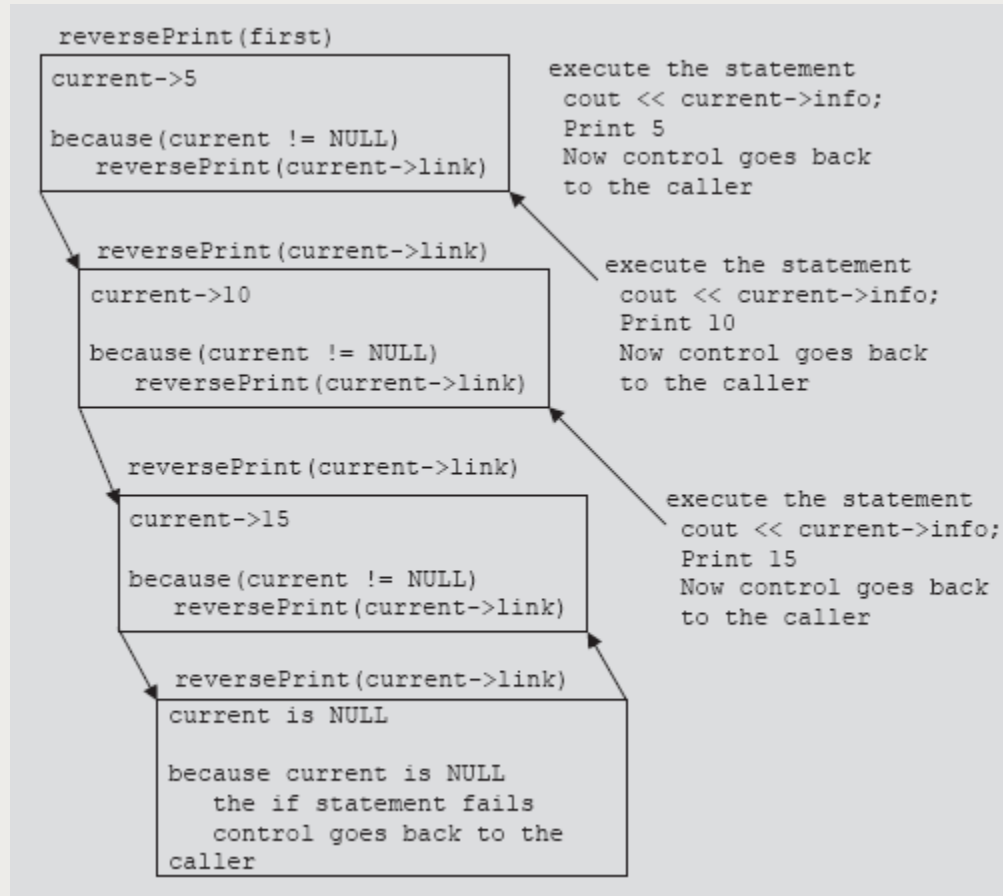
```
if (current != NULL)
{
    reversePrint(current->link);    //print the tail
    cout << current->info << endl;  //print the node
}
```

3.3.3.PRINT A LINKED LIST IN REVERSE ORDER (CONT.)

- Function template to implement previous algorithm and then apply it to a list

```
template <class Type>
void linkedListType<Type>::reversePrint
    (nodeType<Type> *current) const
{
    if (current != NULL)
    {
        reversePrint(current->link);    //print the tail
        cout << current->info << " ";  //print the node
    }
}
```

3.3.3.PRINT A LINKED LIST IN REVERSE ORDER (CONT.)



Execution of the statement `reversePrint(first);`

3.3.3.PRINT A LINKED LIST IN REVERSE ORDER (CONT.)

- The function `printListReverse`
 - Prints an ordered linked list contained in an object of the type `linkedListType`

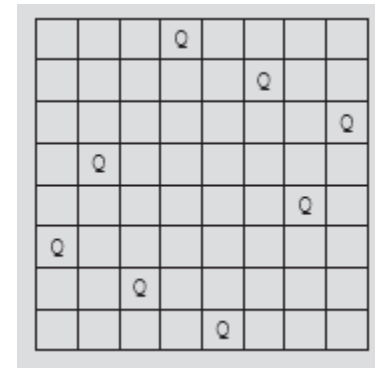
```
template <class Type>
void linkedListType<Type>::printListReverse() const
{
    reversePrint(first);
    cout << endl;
}
```

3.4. RECURSION OR ITERATION?

- Dependent upon nature of the solution and efficiency
- Efficiency
 - Overhead of recursive function: execution time and memory usage
 - Given speed memory of today's computers, we can depend more on how programmer envisions solution
 - Use of programmer's time
 - Any program that can be written recursively can also be written iteratively

3.5.RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE

- 8-queens puzzle
 - Place 8 queens on a chess-board
 - No two queens can attack each other
 - Nonattacking queens
 - Cannot be in same row, same column, same diagonals



A solution to the 8-queens puzzle

3.5. RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE (CONT.)

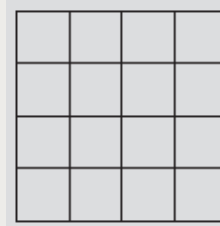
- Backtracking algorithm
 - Find problem solutions by constructing partial solutions
 - Ensures partial solution does not violate requirements
 - Extends partial solution toward completion
 - If partial solution does not lead to a solution (dead end)
 - Algorithm backs up
 - Removes most recently added part
 - Tries other possibilities

3.5. RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE (CONT.)

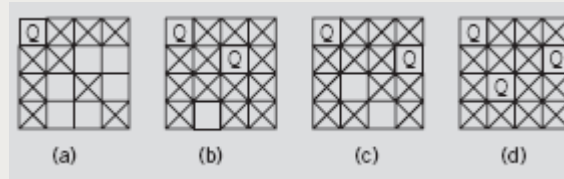
- n -Queens Puzzle
 - In backtracking, solution represented as
 - n -tuple $(x_1, x_2, \dots, x_n)$
 - Where x_i is an integer such that $1 \leq x_i \leq n$
 - x_i specifies column number, where to place the i th queen in the i th row
 - Solution example for Figure 6-11
 - $(4, 6, 8, 2, 7, 1, 3, 5)$
 - Number of n -tuple representing a solution: $n!$

3.5. RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE (CONT.)

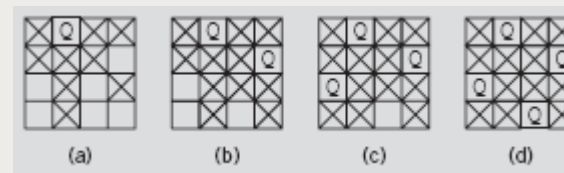
- ***n*-Queens Puzzle (cont.)**
 - 4-queens puzzle



Square board for the 4-queens puzzle



Finding a solution to the 4-queens puzzle



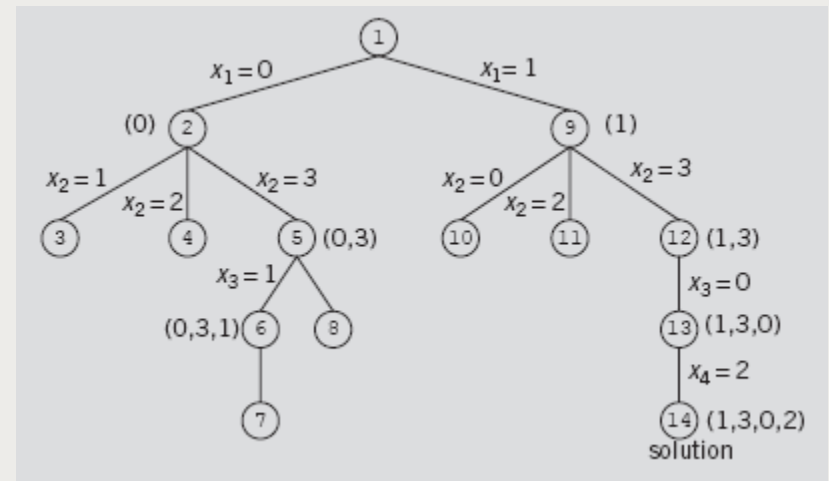
A solution to the 4-queens puzzle

3.5. RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE (CONT.)

Backtracking and the 4-Queens Puzzle

Rows and columns numbered zero to three

Backtracking algorithm can be represented by a tree



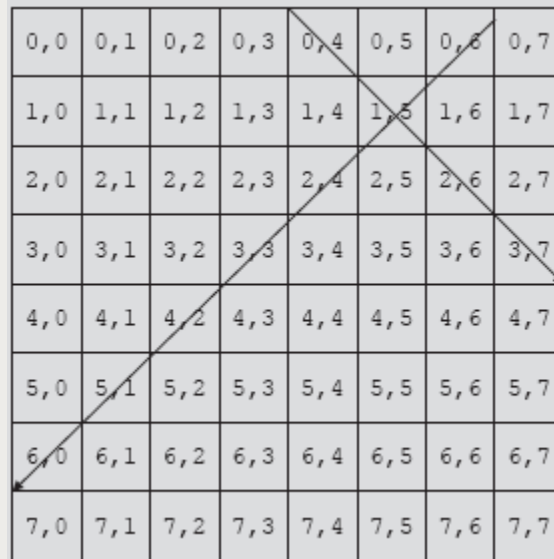
4-queens tree

3.5. RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE (CONT.)

- 8-Queens Puzzle
 - Easy to determine whether two queens in same row or column
 - Determine if two queens on same diagonal
 - Given queen at position (i, j) , (row i and column j), and another queen at position (k, l) , (row k and column l)
 - Two queens on the same diagonal if $|j - l| = |i - k|$, where $|j - l|$ is the absolute value of $j - l$ and so on
 - Solution represented as an 8-tuple
 - Use the array `queensInRow` of size eight
 - Where `queensInRow[k]` specifies column position of the k th queen in row k

3.5. RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE (CONT.)

8-Queens Puzzle (cont.)



An 8x8 grid representing a chessboard. Each cell contains a coordinate pair (row, column) ranging from (0,0) to (7,7). Two diagonals are drawn across the board: a main diagonal from the top-left to the bottom-right, and an anti-diagonal from the top-right to the bottom-left. The cells are shaded in a light gray color.

0,0	0,1	0,2	0,3	0,4	0,5	0,6	0,7
1,0	1,1	1,2	1,3	1,4	1,5	1,6	1,7
2,0	2,1	2,2	2,3	2,4	2,5	2,6	2,7
3,0	3,1	3,2	3,3	3,4	3,5	3,6	3,7
4,0	4,1	4,2	4,3	4,4	4,5	4,6	4,7
5,0	5,1	5,2	5,3	5,4	5,5	5,6	5,7
6,0	6,1	6,2	6,3	6,4	6,5	6,6	6,7
7,0	7,1	7,2	7,3	7,4	7,5	7,6	7,7

8 x 8 square board

3.5. RECURSION AND BACKTRACKING: 8-QUEENS PUZZLE (CONT.)

- 8-Queens Puzzle (cont'd.)
 - General algorithm for the function `canPlaceQueen(k, i)`

```
for (int j = 0; j < k; j++)  
    if((queensInRow[j] == i) //there is already a queen in column i  
        || (abs(queensInRow[j] - i) == abs(j-k)) //there is already  
                                                    //a queen in one of the diagonals  
                                                    //on which square (k,i) lies  
        return false;  
  
return true;
```

SUMMARY

- Recursion
 - Solve problem by reducing it to smaller versions of itself
- Recursive algorithms implemented using recursive functions
 - Direct, indirect, and infinite recursion
- Many problems solved using recursive algorithms
- Choosing between recursion and iteration
 - Nature of solution; efficiency requirements
- Backtracking
 - Problem solving; iterative design technique

Thank You

